

NEMA 23 / MKS SERVO57D — Bring-Up SOP

Arctos Robotic Arm — joint X

September 2026

Caution: This procedure has never been completed on real NEMA 23 hardware

No NEMA 23 joint on this arm has ever communicated over CAN. The MKS SERVO57D fitted to Joint X turned out to be the **RS485 variant**, which has no CAN transceiver, so every step below that involves the CAN protocol is extrapolated from the SERVO42D work rather than confirmed on a 57D.

Each step is tagged **[VERIFIED]** or **[UNVERIFIED]**. **[VERIFIED]** means confirmed on this arm and expected to carry over directly, which is generally the host-side and physical steps. **[UNVERIFIED]** means assumed by analogy with the 42D — plausible, but nobody has seen it work on a 57D. Confirm each of those as you go and update this document with what you actually observe.

Background, protocol details and troubleshooting live in `Arctos_CAN_Interface.pdf`. Bare terminal commands live in `BASIC_CAN_COMMANDS.pdf`.

The joint

Joint	X, base and waist rotation
CAN ID	0x01 per config; confirm by scan
Gear ratio	13.5:1
Motor	NEMA 23, 1.8 N · m, 2.8 A, 6.35 mm shaft
Driver	MKS SERVO57D

Caution: A NEMA 23 is substantially more dangerous than a NEMA 17

1.8 N · m through 13.5:1 is roughly **24 N · m at the joint**, about 75 times the NEMA 17 joints' output torque, and more than enough to injure a hand or destroy a printed part before you can react. Keep clear of the mechanism, keep the emergency stop pasted in a second terminal, and keep a hand on the power switch for every motion step.

Step 0 — Confirm the board is the CAN variant **[VERIFIED]**

Do this first. It is the failure that has already cost this project weeks.

1. Read the part number and silkscreen on the MKS SERVO57D board.
2. Confirm it is the **CAN** variant, not RS485. The two look similar and both present a differential pair.

Caution: RS485 boards will never work, at any bitrate or address

RS485 carries MKS's own async serial protocol, not CAN framing. A CAN controller and an RS485 transceiver cannot talk over the same wires. If this board is RS485 you must either swap it for the CAN variant, which keeps this whole procedure and every script valid, or treat the joint as a separate RS485 subsystem with its own adapter and driver software, which is out of scope here.

Pass: Board is confirmed to be the CAN variant.

If it fails: Stop. No amount of software work will make an RS485 board appear on the bus.

Step 1 — Pre-power electrical checks [VERIFIED]

1. Everything powered off, measure CAN_H to CAN_L. Expect about 60 Ω .
2. Confirm only CAN_H, CAN_L and GND run between the CANable and the controller.
3. **Check the supply.** A NEMA 23 at 2.8 A draws far more than the NEMA 17 joints. Confirm the supply is rated for it and that the wiring to the driver is sized appropriately.
4. Confirm the joint is mechanically clear and that you know its travel limits.

Pass: About 60 Ω , correct wiring, adequate supply.

Step 2 — Configure the controller panel [UNVERIFIED]

The 57D's menu layout has not been confirmed to match the 42D's. Set and **save**:

CAN ID	0x01
CAN Rate	500
CAN Response, shown as CANRSP	ENABLE
CAN CRC	SUM-8
Working current	2.8 A or below, the NEMA 23's rating
Working mode	SR_vFOC — see the caution below

Power-cycle after saving.

Caution: Set the working current deliberately

A 42D in this project was found storing 3000 mA against a 1.2 A motor. On a 2.8 A NEMA 23 the consequences of an over-set current are correspondingly larger.

Caution: Set the working mode to SR_vFOC, not a CR_ mode

Carried over from the 42D procedure and **not yet confirmed on 57D hardware**, like the rest of this step. The controller ships in CR_vFOC. The CR_ modes drive the motor from the physical pulse interface (STP/DIR/En); the SR_ modes drive it from serial and CAN commands, which is how this arm is controlled. `arctos_interface.cpp`'s `setupMotorParameters()` sends SET_WORKING_MODE (0x82) with SR_vFOC (value 5) to every joint.

A mode mismatch **imitates a communication fault**: the controller accepts a frame and acknowledges its checksum without turning the shaft, so a “no motion” result looks like wiring or CAN. Note also that unlike the 42D, the 57D has a real IN_2 input and so does not need limit port remap to get a second endstop — but if remap is used anyway, the same serial-mode restriction applies.

The mode can also be set over CAN rather than on the panel:

```
# SET_WORKING_MODE (0x82) = SR_vFOC (5), CAN ID 0x01
# checksum 0x01 + 0x82 + 0x05 = 0x88
cansend can0 001#820588
```

Pass: Settings survive a power cycle.

If it fails: If the menu differs from the above, record what it actually offers and update this document. That is exactly the unverified part.

Step 3 — Bring up the CAN interface [VERIFIED]

Identical to the NEMA 17 procedure. This is host-side and motor-independent.

```
lsusb
ls -l /dev/serial/by-id/
sudo pkill slcand
sudo ip link delete can0 2>/dev/null
sudo slcand -o -c -s6 /dev/serial/by-id/usb-Openlight_Labs_CANable2* can0
sudo ip link set can0 txqueuelen 1000
sudo ip link set up can0
ip -br link show can0
```

Pass: The last command prints `can0` followed by UP.

Step 4 — Confirm the real CAN ID [UNVERIFIED]

The scan itself is verified; what is unverified is whether a 57D answers the 0x31 query the same way a 42D does.

```
cd ~/arctos_ws/src/arctos_can_control/arctos_can_control
python3 can_id_scan.py --channel can0 --ids 1-16
```

Read-only. It never enables coils and never commands motion.

Pass: Exactly one responder, with a valid checksum.

If it fails: Total silence with the board powered and the wiring confirmed is the RS485 signature. Go back to Step 0 and check the part number before doing anything else. Note that the scan adapter does not populate the kernel's bus-error counters, so you cannot use "zero errors" as corroboration here.

Step 5 — Read-only health check [UNVERIFIED]

```
python3 joint_reliability_test.py --channel can0 --can-id 0x01 \
    --gear-ratio 13.5 --samples 200
```

Query	Opcode	Expected, by analogy
Encoder	0x31	stable; zero jitter, zero drift
Velocity	0x32	zero
Position error	0x39	small
Enable state	0x3A	01
Shaft protection	0x3E	00

Caution: Confirm the encoder resolution on the 57D

This procedure assumes 16384 counts per revolution and 3200 microstep pulses per revolution, as on the 42D. **Verify this before trusting any angle arithmetic:** command a known number of pulses and check that the encoder delta matches. If the 57D differs, every conversion in this SOP changes.

Pass: 100% response, zero checksum failures, zero jitter at rest.

Step 6 — Check the mechanism by hand [VERIFIED]

1. Disable the coils:

```
cansend can0 001#F300F4
```

2. Confirm the enable state now reads 00:

```
cansend can0 001#3A3B
```

3. Turn the joint output by hand, gently, both directions, watching the encoder.

Caution: Do not drop coils on a loaded base joint

Joint X carries the entire arm above it. Removing holding torque may let the whole assembly swing. Only do this with the arm supported or removed.

Pass: The encoder tracks your hand both ways and the motion feels smooth.

If it fails: Gritty resistance means the gear mesh needs a plastic-safe grease. No encoder change while the output turns means the motor is not coupled to it.

Step 7 — First commanded motion [UNVERIFIED]

Paste the emergency stop into a second terminal, ready but unsent, before sending anything:

```
cansend can0 001#F7F8
```

Then one small move:

```
python3 joint_unstick_move_test.py --can-id 0x01 --gear-ratio 13.5 \  
  --step-degrees 0.25 --num-steps 1 --max-unstick 0 \  
  --band-lo -1 --band-hi 1 --speed 25
```

Caution: Start smaller than you would on a NEMA 17

At 13.5:1 the reduction is far lower than the NEMA 17 joints' 48:1 and 67.82:1, so the same commanded joint angle turns the motor far less, and the joint moves much faster for a given motor speed. A 0.25° joint step here is only about 3.4° of motor rotation. Verify direction and magnitude on this first step before increasing anything.

Pass: About 100% of commanded, with FD=[1, 2].

If it fails: Do not retry into a stall. Return to Step 6.

Step 8 — Stepped travel test [UNVERIFIED]

```
python3 joint_unstick_move_test.py --can-id 0x01 --gear-ratio 13.5 \  
  --step-degrees 2.0 --num-steps 5 --max-unstick 0 \  
  --band-lo -15 --band-hi 15 --speed 25
```

Then the same distance back:

```
python3 joint_unstick_move_test.py --can-id 0x01 --gear-ratio 13.5 \  
  --step-degrees -2.0 --num-steps 5 --max-unstick 0 \  
  --band-lo -15 --band-hi 15 --speed 25
```

Pass: Every step 99–101% with FD 02, and the round trip returns to start.

Step 9 — Establish this motor’s speed limit [UNVERIFIED]

The NEMA 17 figures do not transfer. The 57D has different winding inductance and a different torque-speed curve, so this motor’s usable ceiling must be measured rather than assumed. The measured table in `Arctos_CAN_Interface.pdf` is 42D data; do not read Joint B’s numbers across to a 57D.

With the joint unloaded and clear:

```
python3 joint_speed_limit_test.py --can-id 0x01 --gear-ratio 13.5 \
  --arc-lo -12 --arc-hi 12 --band-lo -15 --band-hi 15 \
  --speeds 50,100,150,200,250 --accel 8 --pause-s 15
```

The usable limit is the highest commanded speed where actual rpm still tracks it to within about 1%.

Caution: Keep the duty cycle down

Running a long ladder of trials back to back overheats the motor. Fifteen seconds between trials is reasonable; one second is not. Stop as soon as you have the answer, and put a hand on the motor part way through — there is no temperature opcode, so nothing in software will warn you.

Record the result here once measured:

Tracks commanded within 1% up to	_____ rpm
Roll-off begins	_____ rpm
Saturation ceiling	_____ rpm

Step 10 — Acceptance

Check	Criterion
Board variant	confirmed CAN, not RS485
CAN ID	confirmed by scan
Comms	100% response, zero checksum failures
Encoder at rest	zero jitter, zero drift
Encoder scale	confirmed for the 57D, not assumed
Hand check	smooth, tracks both directions
Per-step accuracy	99–101% of commanded
Status frames	FD 02 on every step
Round trip	returns to start
Speed limit	measured and recorded above

Endstops — wiring and verification [VERIFIED]

The 57D has **two real limit inputs**, IN_1 and IN_2, so unlike the 42D it needs **no limit port remap**. Wire each sensor to EVCC, EGND and its input on the left green block, and set SW1 pins 2 and 3 ON so the board feeds the sensors its own 5 V.

Check the pins with the joint stationary, passing a magnet at each sensor:

```
python3 joint_endstop_test.py --can-id 0x01
```

Pass: Resting byte 0x0F; one sensor drops it to 0x0E (IN_1), the other to 0x0D (IN_2), returning to 0x0F between passes.

If it fails: Nothing moves. An unwired 57D input reads *high*, so a reading of *low* at rest is not “not connected” — suspect a reversed 3-pin plug, which swaps **Signal** and **GND** around the middle +V pin and straps the input to ground while the sensor stays powered and its LED still works.

Caution: EndLimit is not what makes an endstop readable

EndLimit governs only whether the driver *acts* on a limit. 0x34 reports the pin regardless. If an input is not reporting, enabling EndLimit will not change that.

Note which sensor lands on which input before homing. GoHome homes to IN_1; a joint whose only sensor sits on IN_2 cannot be homed by the driver at all. Record the mapping and the joint’s travel with:

```
python3 joint_calibrate.py y
```

Run it twice at the same speed — a single figure is not a result, and approach speed biases the trigger position from both ends at once.

Step 11 — Shutdown [VERIFIED]

Stop any running script with Ctrl+C first.

```
cansend can0 001#F300F4
sudo ip link set can0 down
sudo pkill slcand
```

Then unplug the adapter. Skip the first line if the arm above this joint is not supported and needs it to keep holding.

Update this document

Every step tagged [UNVERIFIED] is a claim nobody has tested on a 57D. When you get CAN-variant hardware onto the bus, work through them in order and replace the tag with what you actually saw, including anything that turned out to differ from the 42D.